

Builder Note — Rahul Sharma

One thing I built, what I owned, a rabbit hole I actually fell into, and how I decide whether an AI result is real.

[Live demo](#)[Source code](#)

1 The thing I built

[Agentic Orchestration](#)[LangGraph](#)[Hybrid ML + Rules](#)[Human-in-the-Loop](#)

AI Cargo — an agentic risk system for pharmaceutical cold-chain shipments. A shipment streams temperature/telemetry windows; the system scores spoilage risk with a hybrid rules + ML engine, then hands medium-and-up events to a **LangGraph agent** that plans and runs eight specialized tools (compliance, cold-storage, rerouting, scheduling, insurance, notification, triage, approvals), reflects on what actually happened, and stops at a human-review gate before anything consequential ships. Every decision writes an immutable, SHAP-annotated audit record.

I picked this project because it's the same problem shape as the product here: **large numbers of tool-using agents that have to be fast, reproducible, and trustworthy — ending in a call of "is this result real or nonsense?"**

Stack: Python, FastAPI, LangGraph, XGBoost + Optuna + SHAP, Supabase (pgvector), React / Vite / Tailwind.

2 What I owned

I owned my slice end to end and worked across every layer of the system:

- **The scoring engine.** 14 engineered features, 8 deterministic safety rules, and an XGBoost model tuned with Optuna (30 trials, optimized for **PR-AUC** because spoilage is rare — `scale_pos_weight ≈ 5`). Split by shipment, not by row, so I wasn't leaking the future into training.
- **The fusion layer** that combines rules and ML into one score and risk tier (see section 4).
- **The orchestration graph** — interpret → plan → execute → observe → reflect → revise → human-review — with a deterministic fallback path so the system still runs with **zero LLM** if a provider is down.
- **The agent tools and cascade** that passes one tool's output into the next, plus the FastAPI backend (25 endpoints + WebSocket) and the React dashboard a non-technical ops user actually clicks through.

3 A recent technical rabbit hole

My reflect → revise loop kept **re-triggering tools that had already succeeded**, and occasionally proposing tools that were never needed. On the dashboard it looked like the agent was "doing something," so it was easy to miss — classic *technically functional but untrustworthy*. I instrumented every node's input/output and traced it: the LLM reflection step was flagging **optional** tools (rerouting, triage) as "gaps," and revise dutifully re-planned them. The model wasn't wrong exactly — it was helpfully hallucinating work. Two fixes:

1. **A hard filter in code, not in the prompt.** Revise can only emit tools in `mandatory_set` or `failed_set`; anything else is rejected before execution. I stopped trusting the prompt to constrain behavior the code could guarantee.
2. **Redesigned to "act-first, always-review."** Medium-and-up events execute once, then *observe* the real results, then reflect on those — and the human gate fires every time. That killed the phantom re-execution loop and made runs reproducible.

The lesson I keep relearning: **an agent that looks busy is not the same as an agent that's correct**, and the cheapest guardrail is usually deterministic code wrapping the model, not a longer prompt.

4 How I'd sanity-check an AI result

This is the part I care about most, and it's baked into AI Cargo rather than bolted on:

1. **Never let the model be the only vote.** Final risk = $0.4 \cdot \text{rules} + 0.6 \cdot \text{ML}$, with a **deterministic veto**: if a hard safety rule fires above threshold, no benign ML prediction can talk the score back down.
2. **Make "unknown" loud, not silent.** A missing/NaN score doesn't default to "safe" — it defaults to **MEDIUM (escalate)**. The dangerous failure is confident silence.
3. **Demand an explanation you can inspect.** Every prediction logs its **top-5 SHAP drivers**. If the "why" is nonsense, I distrust the "what," even when the number looks plausible.
4. **Ask whether the effect could be noise.** I read **PR-AUC and recall-at-fixed-precision**, not accuracy, and split by entity to rule out leakage. Applied to conversion work: an apparent lift gets the same treatment — check power/MDE, segment it, and rule out an instrumentation or assignment bug *before* believing the number.
5. **Constrain the generative parts.** LLM tool-calls are schema-validated and hard-filtered; a malformed or out-of-scope response is rejected and retried, never executed blindly.
6. **Put a human on the consequential path.** Anything medium-and-up pauses for review with the full reasoning trace attached — automation earns autonomy by being right over time, not by default.

I use AI tools aggressively every day — but the moment a result would *act* on the world, I want a rule, a metric, or a human between the model and the consequence.

Happy to walk any of this live — the demo is up, and I can screen-share the orchestration trace for a real shipment.